

High-Density Salt and Pepper Noise Removal from Colour Images by Introducing New Enhanced Filter

Shounak Banerjee

Department of Computer Science
and Engineering,

Techno International New Town
shounak.banerjee.2017@cse.tict.edu.in

Debarpito Sarkar

Department of Computer Science
and Engineering,

Techno International New Town
debarpito.sarkar.2017@cse.tict.edu.in

Debraj Chatterjee

Department of Computer Science
and Engineering,

Techno International New Town
debraj.chatterjee@tict.edu.in

Sunanda Roy Chowdhuri

Department of Computer Science
and Engineering,

Techno International New Town
roychowdhurisunanda9@gmail.com

Abstract: Any image captured by a camera or scanner contains some unwanted particles, which are known as **noise or salt and pepper**. They are black and white color dots that are present in some pixels. The value of those pixels is either 0 or 255. There exist different predefined methods to remove salt and pepper or noise from an image in Matlab. This paper presents an effective salt and pepper filtering and removal algorithm, which has accuracy greater than some of the predefined filters in Matlab like Median Filter, Adaptive Median Filter, MDBUTMF, MDBPTGMF, and Adaptive Weighted Median Filter. The **Peak Signal to Noise Ratio (PSNR) value** for each algorithm has been compared with this proposed algorithm. A comparative analysis has been carried out in this paper between these predefined methods and the proposed algorithm. The tests have been done on few images and the results have been displayed in tabular format.

Keywords: Salt and Pepper, Peak Signal to Noise ratio, Mean-Squared Error.

I. INTRODUCTION

An Image can be defined as a two-dimensional function, $f(x, y)$, which is formulated as $I = f(x, y)$ (1) where x, y are spatial coordinates and I is the grey value or Intensity at that point. The value of $f(x, y)$ at any point gives the Pixel (smallest element of an image) value at that point of an image. When all pixels of an image are with finite and discrete values for its grey level that is output from its $f(x, y)$, that image is called a digital image. It can be a vector or raster type depending on whether the image resolution is fixed or not. In the case of a raster-type digital image, there is a fixed number of rows and columns of a pixel. It can be created from various input devices like digital cameras, scanners, airborne radar, etc. The digital image is represented by a single two-dimensional integer array for a greyscale image and a series of three 2-dimensional arrays for each color band.

Image processing is a method of enhancing an image or extracting useful information from it through some operations or algorithms. It is one type of signal processing where the input is an image and the output may be an enhanced image or some features regarding that image. The most important step to enhance an image is noise removal. Noise is a random variation of unwanted color brightness of information in images. It is produced by the image sensor of

a camera or scanner. More noise in an image, less is the information present. This proposed method is about the removal of salt and pepper noise using a new enhanced image processing algorithm. Salt and Pepper noise is also known as impulsive noise. An image containing salt and pepper noise will have dark pixels in bright regions and white pixels in dark regions as seen in Fig. 1.



Fig. 1. Salt and Pepper Noise (70 % Noise Density)

All colors that exist in a single pixel of an image are represented by a number which is also called pixel value. The range of this value is from 0 to 255 out of which zero is for black and 255 is for white. These 0 and 255 values are the noise elements which has to be removed. The main issue that arises here is to prevent the loss of image edge and texture details. For this reason, algorithms like Modified Decision Based Unsymmetric Median Filter (MDBUTMF), Modified Decision Based Partially Trimmed Global median Filter (MDBPTGMF), etc. are used. In these algorithms, a fixed 3 X 3 matrix is taken at first and then a search for a corrupted pixel is done. If found then it is replaced by either the mean, the median, or trimmed value of the pixels present in that matrix. These algorithms have high accuracy when dealing with less noise. When the noise density increases, these algorithms fail to prevent image data loss. The proposed method overcomes this issue and prevents data loss. This algorithm supports feature extraction by resizing the image accordingly. This is a huge advantage for those who want to do machine learning using images.

Many researchers have designed different algorithms to remove salt and pepper over the years. Many have achieved better accuracy than the predefined ones some of which are discussed here. Pranay Yadav et. al. proposed an algorithm that replaces the noisy pixels with the trimmed values [1]. If the selected window contains all 0 or 255 values then the processing pixels are replaced with the mean of the remaining pixels. Sujaya Kumar Sathua et. al. proposed an algorithm that also follows the same approach, i.e., selecting a 3 X 3 window and the central pixel is considered as the pixel to be processed. If that pixel is found corrupted then the window size is increased according to the algorithm [2]. In the end, the processing pixel is replaced by either the mean, the median, or the trimmed value of the elements present in the current window.

The proposed algorithm takes a noisy image as input. The image is a 3-dimensional matrix, so it is divided into three two-dimensional matrices called red, green, and blue matrix. These three matrices represent the red, green, and blue channels of an RGB image. The proposed algorithm is implemented separately on each of these three matrices and the resultant processed matrices are combined to form the resultant 3d matrix or the output image.

The outline of the paper is as follows. The proposed algorithm is explained in Section 3. An illustration of the proposed algorithm is shown in Section 4. The comparative

analysis between this proposed algorithm and the other algorithms is shown in Section 5. The conclusion is given in Section 6.

II. PROPOSED METHOD

In the proposed algorithm, firstly an Image is taken as input. The image is represented by a three-dimensional matrix which then is divided into three two-dimensional matrices which are called the red, green, and blue channels. From each matrix, a 3 X 3 matrix is selected, and count the number of corrupted pixels in that matrix. The corrupted pixels can be identified by 0 or 255. If all the pixels in the matrix are corrupted, then there exist two outcomes. If the matrix is the first 3 X 3 matrix from each of the red, green or blue matrix, then all pixel value is set to zero. If not, then the corrupted pixels are replaced by the mean of the uncorrupted pixels in the previous 3 X 3 matrix. Now if all the pixels in the matrix are not corrupted, then the corrupted pixels are replaced by the mean of the remaining uncorrupted pixels. This entire process continues until the entire matrix is traversed. After that, the three modified red, green, and blue matrices are merged, and the output image is formed. The algorithm and flowchart of the proposed algorithm are given in the upcoming part.

A. Flowchart

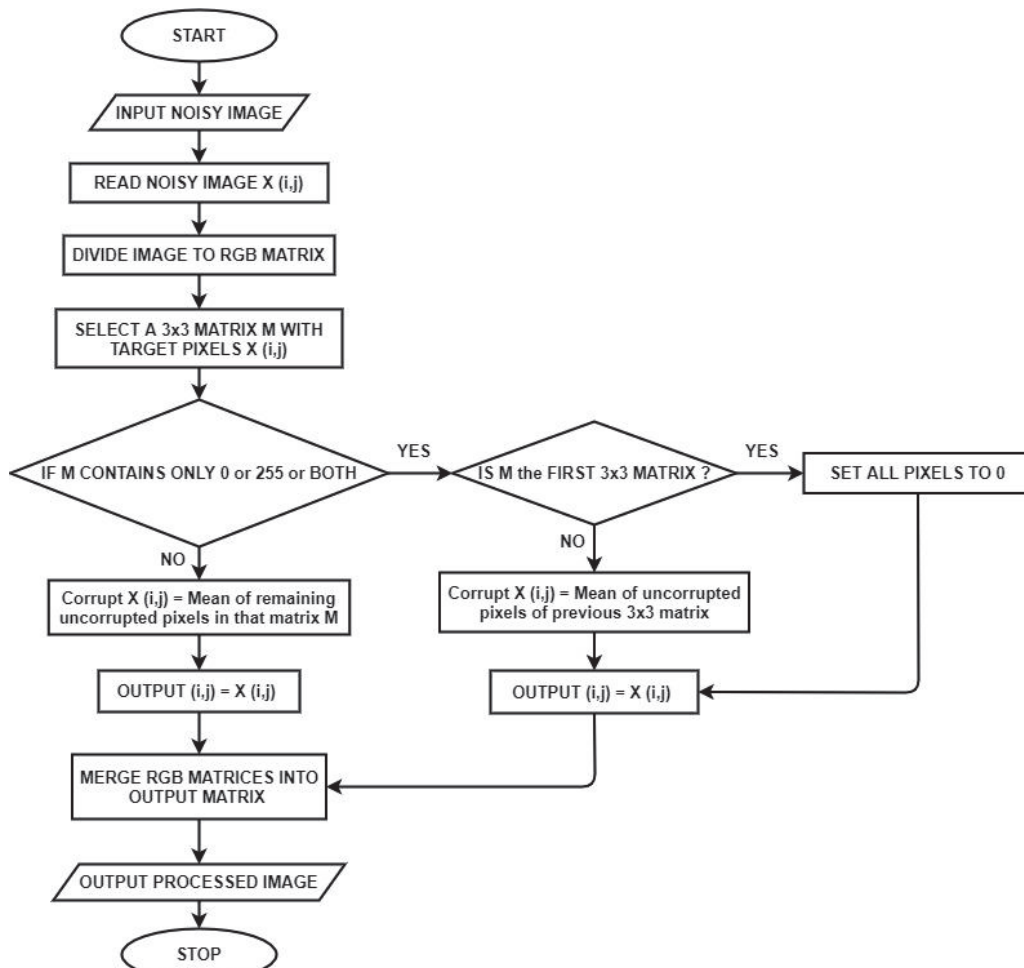


Fig. 2. Flow-Chart diagram of Proposed Algorithm (Modified 3D Median Filter)

B. Algorithm

Step 1: Start.

Step 2: At first an image is taken, and fixed salt and pepper noise is applied to it.

Step 3: Now the noisy image is taken as input and it is converted into its 3-dimensional matrix form.

Step 4: Now the 3-dimensional matrix is broken down into three 2-dimensional matrices named Red, Green, and Blue. These three matrices represent the Red-Green-Blue Channel of a color image. Here is a note. When an image is segregated to RGB channels then it segregates in this order Red, Blue, Green, and not Red, and Green, and Blue.

Step 5: The first channel matrix is taken which is Red.

Step 6: Now a 3x3 matrix is selected and then another condition arises. Are all pixel values in that matrix 0 or 255 or both?

Case 1: If yes then another condition arises. Is M the first 3x3 matrix?

Case 1.1: If yes then all the pixel values are set to zero and the control jumps to Step 7.

Case 1.2: If not then the corrupted pixels are replaced by the mean of uncorrupted pixels of the previous 3x3 matrix, and the control jumps to Step 7.

Case 2: If not then the corrupted pixels are replaced by the mean of the remaining uncorrupted pixels of that matrix.

Step 7: Continue Step 6 till the entire channel matrix is traversed.

Step 8: Repeat Steps 6 to 7 for the other two channels.

Step 9: Merge the three-channel matrices to create a single 3-dimensional matrix that represents the output image.

Step 10: The processed image is sent as output.

Step 11: Stop.

III. ILLUSTRATION OF PROPOSED ALGORITHM

The proposed algorithm works on three different cases which are explained below with the help of an example. A random 9x9 matrix is taken as an example.

Case 1: When all pixels of the selected 3x3 matrix are either 0 or 255 or both and that selected 3x3 matrix is the first one. The selected matrix is highlighted in bold line.

0	255	0	0	98	255	45	65	0
255	0	0	238	12	99	12	23	99
0	255	255	46	0	255	99	34	99
99	123	34	123	99	34	34	67	255
99	255	34	0	45	56	0	255	112
99	0	231	43	255	99	255	99	58
99	123	0	234	123	23	35	0	67
99	255	99	99	0	23	99	78	97
0	99	99	255	99	253	56	255	99

Here all the pixel values in the first selected matrix are either 0 or 255. For this case, all the pixel values are set to zero.

0	0	0	0	98	255	45	65	0
0	0	0	238	12	99	12	23	99
0	0	0	46	0	255	99	34	99
99	123	34	123	99	34	34	67	255
99	255	34	0	45	56	0	255	112
99	0	231	43	255	99	255	99	58
99	123	0	234	123	23	35	0	67
99	255	99	99	0	23	99	78	97
0	99	99	255	99	253	56	255	99

Case 2: When all pixels of the selected 3x3 matrix are either 0 or 255 or both and that selected 3x3 matrix is not the first one. The selected matrix is highlighted in bold line.

63	0	0	0	255	255	45	65	0
121	0	0	0	255	0	12	23	99
64	255	255	255	0	0	99	34	99
99	123	34	123	99	34	34	67	255
99	255	34	0	45	56	0	255	112
99	0	231	43	255	99	255	99	58
99	123	0	234	123	23	35	0	67
99	255	99	99	0	23	99	78	97
0	99	99	255	99	253	56	255	99

Here all the pixel values in the selected matrix are either 0 or 255. For this case, the corrupted pixels are replaced by the mean of uncorrupted pixels of the previous 3x3 matrix [63 121 65, 0 0 255, 0 0 255]. The mean of 63,121 and 64 is 83.

63	0	0	83	83	83	45	65	0
121	0	0	83	83	83	12	23	99
65	255	255	83	83	83	99	34	99
99	123	34	123	99	34	34	67	255
99	255	34	0	45	56	0	255	112
99	0	231	43	255	99	255	99	58
99	123	0	234	123	23	35	0	67
99	255	99	99	0	23	99	78	97
0	99	99	255	99	253	56	255	99

Case 3: When only a few pixels of the selected matrix are corrupted.

63	0	0	83	83	83	45	65	0
121	0	0	83	83	83	12	23	99
65	255	255	83	83	83	99	34	99
99	123	34	123	99	34	34	67	255
99	255	34	0	45	56	0	255	112
99	0	231	43	255	99	255	99	58
99	123	0	234	123	23	35	0	67

99	255	99	99	0	23	99	78	97
0	99	99	255	99	253	56	255	99

Here there exist four corrupted pixels and 5 uncorrupted pixels. In this case, the algorithm replaces the corrupted pixels with the mean of the remaining uncorrupted pixels (in this case 34, 67, 112, 99, and 58). In this case, the mean is 74. The resultant matrix is given below.

63	0	0	83	83	83	45	65	0
121	0	0	83	83	83	12	23	99
65	255	255	83	83	83	99	34	99
99	123	34	123	99	34	34	67	74
99	255	34	0	45	56	74	74	112
99	0	231	43	255	99	74	99	58
99	123	0	234	123	23	35	0	67
99	255	99	99	0	23	99	78	97
0	99	99	255	99	253	56	255	99

IV. COMPARATIVE ANALYSIS AND RESULTS

For testing, color images of Lena and Satoru Gojo are taken. Salt and Pepper noise is added to these images.

TABLE I. PSNR VALUES FOR DIFFERENT NOISE DENSITY

Algorithms	PSNR in dB								
	Noise Density								
	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %
MF	36.6145	34.646	32.1284	28.8066	24.81	20.6115	16.7564	13.6082	11.1638
AMF	38.0372	36.1205	34.211	32.7681	31.6704	30.6476	29.9982	23.9258	16.6012
MDBUTMF	45.2992	40.8584	41.2884	40.2236	38.9578	37.8056	36.648	35.1943	28.8996
PSMF	21.6023	18.5721	16.7072	15.255	13.988	12.8086	11.729	10.7443	9.8729
BPDF	47.7339	43.7524	40.8084	38.4247	36.0544	33.5662	30.5916	26.2819	16.9140
PA	45.2394	42.1972	40.3489	38.9343	37.7167	36.5472	35.1904	33.2965	30.1486

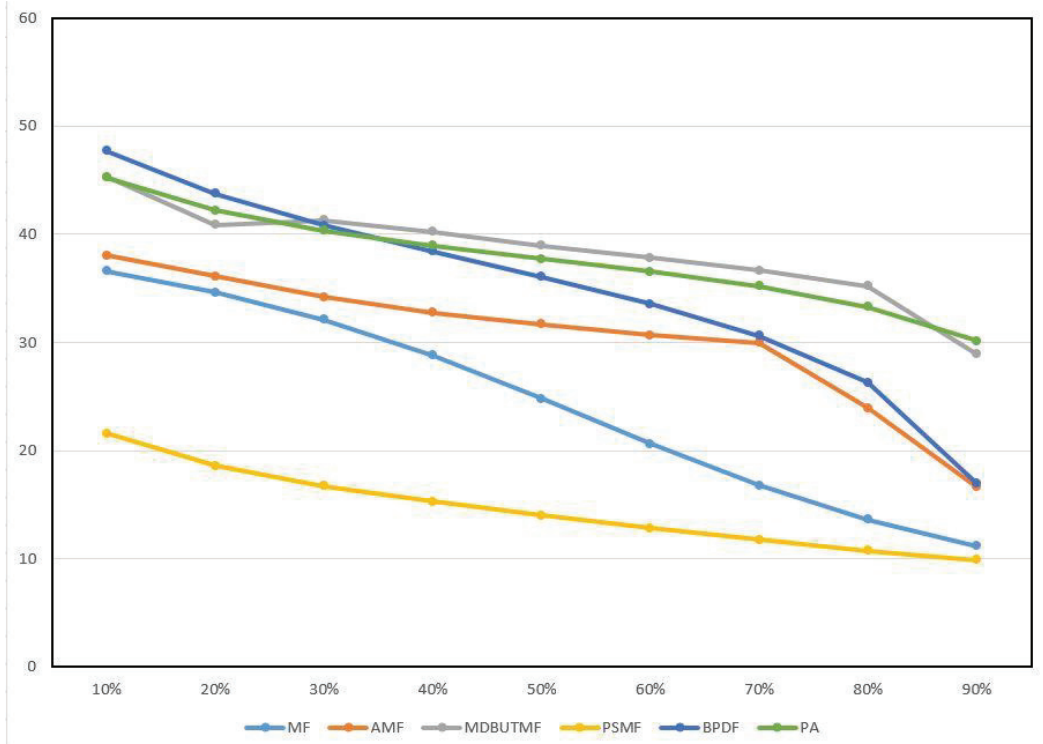


Fig. 3. PSNR (in dB) vs. Noise Density

Variation in noise density is kept from 10 % to 90 %. Then the noise is removed using the proposed algorithm and other existing algorithms like Median Filter (MF), Adaptive Median Filter (AMF) [3], Modified Decision Based Unsymmetric Trimmed Median Filter (MDBUTMF) [4], Progressive Switching Median Filter (PSMF) [5] and Based on Pixel Density Filter (BPDF) [6]. The execution has been done in a 2.8 GHz, 8 GB RAM, Nvidia GTX 1050 system, and MATLAB 2018. The performance of the existing algorithms and the proposed algorithm has been measured using Mean-Squared Error (MSE) (1) and Peak Signal to Noise Ratio (PSNR) (2).

The formula of Mean-Squared Error is,

$$MSE = \frac{1}{m \times n} \sum_{i=1}^m \sum_{j=1}^n \{Y(i, j) - \hat{Y}(i, j)\}^2 \quad (1)$$

The formula of PSNR is,

$$PSNR = 10 \log_{10} \frac{255^2}{MSE} \quad (2)$$

The PSNR values of the proposed algorithm and the existing algorithms are compared for noise density varying from 10 % to 90 %. The results are displayed in Table 1 and Fig 3.

From Fig. 2 and Table 1, it is evident that the algorithm MDBUTMF has better filtering results than the proposed algorithm when noise density varies from 30% to 80%. But its time complexity is much greater than that of the proposed algorithm. The other algorithms have lower filtering results concerning the proposed algorithm. From Fig. 2 it is also evident that the proposed algorithm offers the best result when noise density is 90 %.

The PSNR and MSE values concerning the varying noise density for the proposed algorithm are given in Table 2.

The proposed algorithm shows better results at a noise density $\geq$ of 50 % when compared with the above five existing algorithms. The restored images obtained at different noise density values by the proposed algorithm are shown in Fig. 4.

TABLE II. PSNR AND MSE VALUES FOR DIFFERENT NOISE DENSITY

Noise Density in %	PSNR value	MSE value
10	45.2394	5.9011
20	42.1972	11.8005
30	40.3489	18.1995
40	38.9343	25.0726
50	37.7167	33.0362
60	36.5472	43.1735
70	35.1904	59.7264
80	33.2965	91.9021
90	30.1486	189.8975



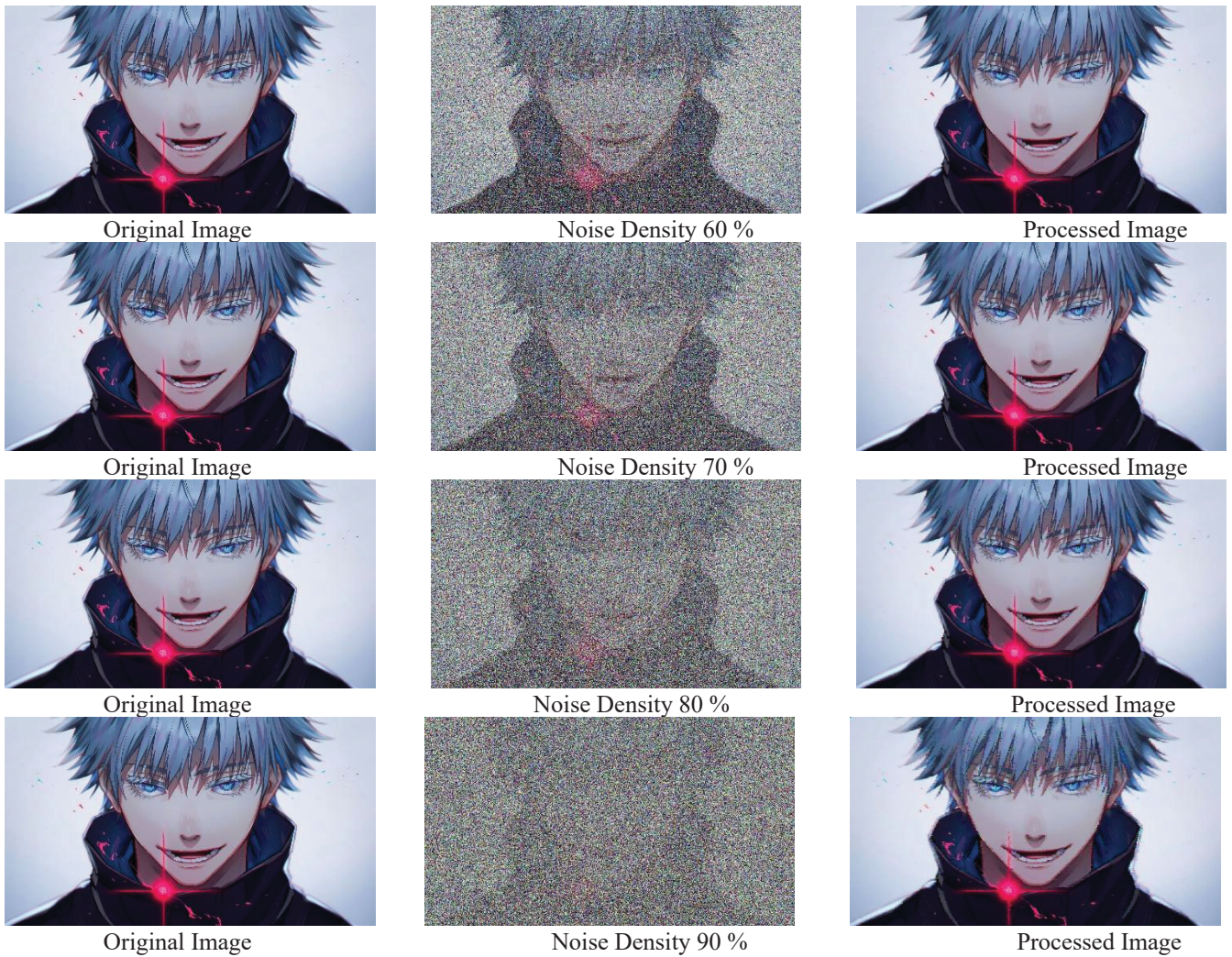


Fig. 4. Experiment Results of the proposed method on an Anime image with noise density varying from 10-90%.

From Fig. 4 it is evident that the proposed algorithm efficiently removed salt and pepper noise from the image even when the noise density is 90% indicating a higher accuracy rate.

V. CONCLUSION

In this paper, a comparative analysis has been carried out between different existing filtering algorithms and a new proposed algorithm. This proposed filtering algorithm aims to remove salt and pepper noise from an image by replacing the noisy pixels with the mean of the uncorrupted pixels. The algorithm works best on both color images and greyscale images. In this paper, the algorithm is tested on a color image and the results have been displayed. The results showed that this algorithm has higher PSNR and MSE values for images with a noise density $\geq$ of 50 %, compared to other existing algorithms. For noise density $<$ 50 %, the algorithm showed similar PSNR and MSE values compared to other existing algorithms. This algorithm has less time complexity compared to other existing algorithms. This algorithm also supports feature extraction by resizing the

[7] [m/matlabcentral/fileexchange/65865-bpdf-for-salt-and-pepper-noise-removal](https://www.mathworks.com/matlabcentral/fileexchange/65865-bpdf-for-salt-and-pepper-noise-removal).

image accordingly. This is a huge advantage for those who want to do machine learning using images.

REFERENCES

- [1] P. Yadav, V. Kumar, M. Jain, A. Samadhiya and S. Jain, "Image De-Noising For Salt and Pepper Noise by Introducing New Enhanced Filter," in International Conference on Innovations in Engineering and Technology, Bangkok, 2013.
- [2] S. K. Sathua, A. Dash and A. Behera, "Removal of Salt and Pepper noise from Gray-Scale and Color Images: An Adaptive Approach," International Journal of Computer Science Trends and Technology, vol. 5, no. 1, pp. 117-126, 2017.
- [3] K. Kintali, "MathWorks," [Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/30068-adaptive-median-filter-matlab-code>.
- [4] S. Kamyra, "MathWorks," [Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/46563-mdbutmf>.
- [5] K. Keizer, "MathWorks," [Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/21757-progressive-switching-median-filter>.
- [6] U. Erkan, "MathWorks," [Online]. Available: <https://www.mathworks.com>